

Redes - FAMAf - Lab 3 — Entrega final

Qué tenés que entregar

Dónde	Qué
Repo (redes26lab3gXX)	Código OMNeT++, network.ned, omnetpp.ini, informe, README.md
Tags	v0.1-parte1 (Parte 1) y v1.0-final (entrega final)
Informe	Markdown en el repo: Parte 1 + Parte 2 + tabla Competición + anexo IA

Recomendaciones antes de entregar

1. **Código** — compila y corre al menos Competicion y un caso con/sin control.
2. **Repo** — todo en main/master, git push, tags creados y pusheados.
3. **README** — cómo compilar y correr con los tags (plantilla abajo).
4. **Informe** — completar secciones e **interpretar** cada figura.
5. **Competición** — tabla con 4 métricas + comando exacto de la corrida.
6. **Releer** — checklist de abajo antes de cerrar.

Checklist de entrega (repo)

Git y tags

- ☐ Último código en **main** o **master** (no solo en otra rama).
- ☐ Push final hecho.
- ☐ Tag **v0.1-parte1** → commit con Parte 1 (Caso1, Caso2).
- ☐ Tag **v1.0-final** → commit de **esta entrega** (Parte 2 + informe final).

Código mínimo en v1.0-final

- ☐ TransportTx.cc, TransportRx.cc en el Makefile.
- ☐ Parte 1 sigue: red sin transporte + [Config Caso1] y [Config Caso2].
- ☐ Parte 2: red con feedback + Caso1_Control, Caso2_Control, **Competicion**.
- ☐ make sin errores

README.md (raíz del repo)

- ☐ Integrantes + ruta del informe (Informe.md, etc.).
- ☐ Cómo compilar.
- ☐ Cómo correr Parte 1 y Parte 2 con tags (copiar bloque de abajo).
- ☐ Comando exacto de **Competición**.

Informe en el repo

- ☐ Archivo fácil de encontrar, con **integrantes**.
- ☐ **Anexo IA** (herramientas, para qué, prompts relevantes, cómo validaron).
- ☐ Gráficas en el repo si no están embebidas en el .md.

Informe — qué no puede faltar

Estructura (7 secciones, puede ser breve)

1. **Título** + integrantes
2. **Resumen** — un párrafo (Parte 1 + qué hicieron en Parte 2 + resultado principal)
3. **Introducción** — Caso 1 = flujo (rxBuffer); Caso 2 = congestión (queue)
4. **Métodos** — algoritmo (rwnd, cwnd, feedback, congestión) con **trozos de código** si ayuda
5. **Resultados** — curvas **interpretadas** + comparación sin/con control
6. **Discusión** — limitaciones y trade-offs (no repetir solo las gráficas)
7. **Referencias** + **anexo IA**

Las 9 preguntas (contestar en el texto)

1. ¿Cómo implementaron rwnd, cwnd y effectiveWindow?
2. ¿Qué información viaja en el feedback?
3. ¿Qué condición usa el transmisor para enviar o esperar?
4. ¿Cómo detectan congestión?
5. ¿Cómo cambia cwnd con y sin congestión?
6. ¿El algoritmo funciona igual en Caso 1 y Caso 2? ¿Por qué?
7. ¿Logra reducir o eliminar pérdidas?
8. ¿Qué costo tiene? (demora, oscilaciones, subutilización del enlace)

9. ¿Qué limitaciones tiene y cómo mejoraría?

Gráficas

Revisen cada figura antes de entregar: **título**, **leyenda** (qué curva es cada cosa) y **ejes con unidades**; debajo, una oración que diga qué concluyen y cómo se relaciona con el análisis (Caso 1/2, con/sin control, una afirmación del texto). Si una gráfica no aporta a ninguna conclusión, **sáquenla**; prioricen las obligatorias (las cuatro curvas de carga, buffers/pérdidas/ventanas y lo que respalde la tabla de Competición). Cada afirmación fuerte en el texto debe citar la figura, valor o tabla que la respalda.

Mínimo de resultados

- Curvas carga ofrecida vs útil: Caso 1 y 2, sin y con control (**4 curvas**).
- Ocupación de buffers, pérdidas, ventanas (rwnd, cwnd, inFlight).
- **Tabla Competición** (abajo).

Competición (obligatorio en el informe)

Comparación entre grupos en **un mismo escenario**. No reemplaza el informe.

- Config: [**Config Competicion**] en omnetpp.ini.
- **No cambien** los valores del enunciado (solo adaptar nombres de red/módulos si su network se llama distinto).
- Correr 200 s y reportar **un valor** por métrica:

Métrica	Cálculo
goodput (pkt/s)	paquetes en Sink / 200
lossRate (%)	100 × descartados / generados
avgDelay (s)	demora E2E promedio (Sink)
stability (pkt)	desvío estándar de effectiveWindow

Tabla en el informe:

Grupo | goodput (pkt/s) | lossRate (%) | avgDelay (s) | stability (pkt)

Incluir el **comando** usado (ej. ./lab3-... -u Cmdenv -c Competicion).

Checklist del informe (última pasada)

- ☐ Caso 1 vs Caso 2 explicados en la intro (flujo vs congestión).
- ☐ Las 9 preguntas respondidas.
- ☐ Sin/con control comparado en ambos casos.
- ☐ Cada figura interpretada; cada afirmación con evidencia.
- ☐ Tabla Competición + comando.
- ☐ Discusión con limitaciones honestas.
- ☐ Anexo IA.

¿En qué se evalúa? (resumen)

- Código legible y modelo correcto (paquetes, buffers, tasas).
- Métricas registradas y **reproducibles** desde el repo.
- **Calidad del análisis**, no cantidad de gráficas sin texto.
- Algoritmo **coherente** entre código e informe.
- Competición bien reportada.
- Discusión honesta si algo no funcionó.

Errores que más nos frenan al corregir

- Entrega solo en rama secundaria, sin merge a main.
- Sin tags o tags viejos que no coinciden con el código.
- Informe sin interpretar gráficas ("aca está la figura").
- Cambiar parámetros de Competicion para inflar la tabla.
- Código en un commit e informe en otro, sin tag claro en v1.0-final.

Dudas de fondo: *enunciado lab3-enunciado.pdf* y *aula virtual*.